informs
https://pubsonline.informs.org/journal/opre

OPERATIONS RESEARCH

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *Operations Research*

ISSN 0030-364X, EISSN 1526-5463

Robust Candidate MaxWeight Certificates for Heterogeneous Compute Scheduling: A Scheduleurm Case Study

(Authors' names are not included for peer review)

Abstract. Modern compute clusters serve jobs whose rates depend on co-location, host load, accelerator memory, and placement. We formulate a configuration-action stochastic processing network and specialize approximate MaxWeight drift accounting to candidate restriction, conservative service estimates, bounded penalties, and optimization error. Positive residual slack yields finite-horizon expected-backlog bounds and a finite-set Foster recurrence certificate. A finite-feature partition supplies a conditional cardinality and support-loss bound; diagonal scaling handles heterogeneous service units, and a variable-duration result states the corresponding frame conditions. Lean formalizes these implications. Scheduleurm evaluates the framework using measured lower-service actions, exact maximization of a canonical frame score, and causal arrival replay. On 477 online traces where all registered policies completed, legacy-to-candidate geometric-mean ratios were 1.574753 for makespan and 2.360104 for mean flow. Of 480 registered online traces, 3 were excluded from these ratios because at least one policy left an unfinished tail. Dedicated-action stability certificates and finite-trace performance evidence remain separate model instantiations. The results identify both the utility of explicit loss accounting and the modeling obligations required to certify a measured scheduler.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

Subject classifications: Queues: scheduling; stochastic models: processing networks; computing: resource allocation

Area of review: Stochastic Models; Computing

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

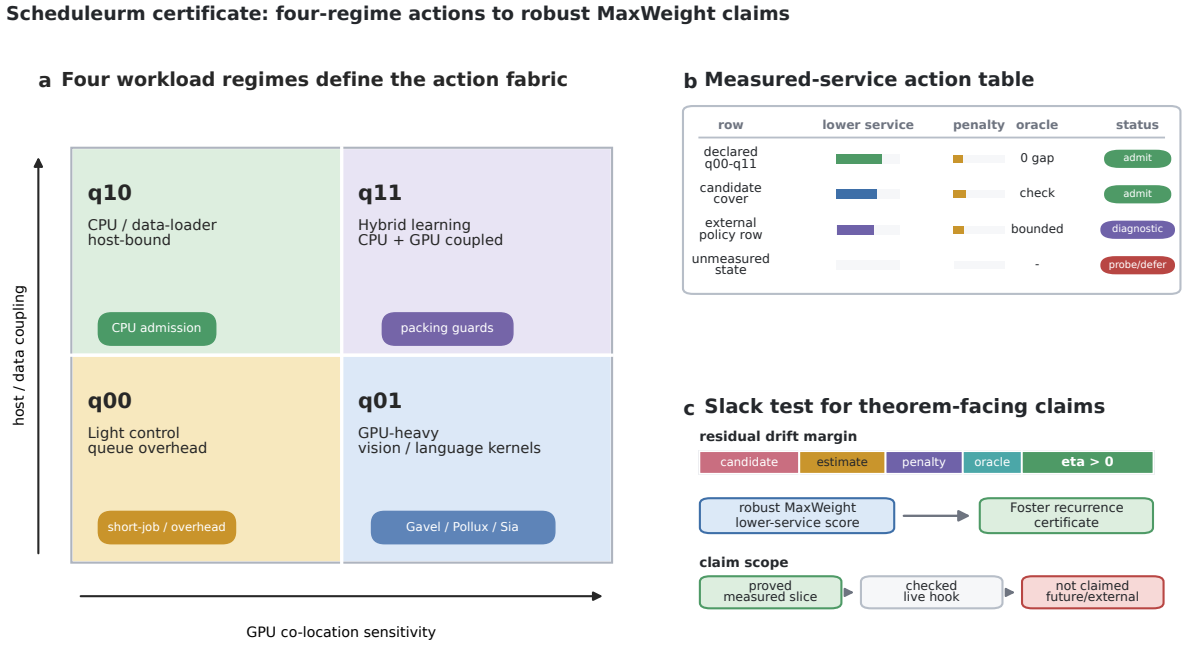


Figure 1 Overview of the robust candidate MaxWeight certificate. A combinatorial full action space is represented by measured finite candidate actions across four workload quadrants. The scheduler scores candidates using the queue-weighted lower-service objective minus bounded penalties, and the stability certificate requires the residual slack to exceed candidate-cover loss, service-estimation error, penalty growth, and approximate-oracle error. External policy families are diagnostic finite actions on the same service cache, not universal full-stack baselines. The measured-service action table summarizes lower-service rows, bounded penalties, oracle gaps, and admission status.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked "sweet spots" can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack. Figure 1 gives the one-page certificate structure: candidate construction, lower-service scoring, slack accounting, and bounded claim boundaries are checked as separate objects.

The theoretical result specializes approximate MaxWeight drift accounting to measured configuration actions. The theorem states exactly how candidate approximation, conservative service estimation, scheduling penalties, and approximate optimization consume capacity slack. When the remaining margin is positive and the stochastic primitives have bounded conditional second

moments, the resulting policy satisfies finite-horizon expected-backlog bounds and a finite-set Foster recurrence certificate. A finite feature partition provides a candidate family whose size is bounded by the number of active cells and whose support loss is at most $L\rho$, given certified representatives and radii. The stability result is stated for both fixed feasible families and state-dependent feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and measured capacity boundaries. For migration and other event-driven actions, a separate variable-duration frame theorem preserves the same per-time slack accounting in cumulative service units and gives an embedded-chain recurrence certificate.

The empirical contribution is a theorem-condition evaluation, not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in Scheduleurm, an existing scheduler used for local research workloads, and report bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit replay compares robust candidate actions with legacy rules and diagnostic policy families on a common measured service cache, so all policies are evaluated against the same lower-service object. Third, holdout diagnostics, finite lower-service capacity certificates, ablation, production coverage, and live-state dry-run trace checks identify which assumptions are certified and which remain finite-slice claims. These results connect the proof obligations to scheduler artifacts; they do not replace a universal fabric-cover theorem or an exhaustive full-stack scheduler competition. Frozen FJSP, MMRCPSP, and port holdouts additionally test whether the finite trajectory-action interface survives domain-specific feasibility and reconfiguration constraints; they are portability evidence, not automatic transfer of the server stochastic model.

The paper is organized as follows. We first state the claim scope and evidence contract, then position the work relative to scheduler and queueing literatures. We then introduce the configuration-action processing network, give the candidate approximation and robust MaxWeight stability theorem, describe the Scheduleurm instantiation, report the computational evidence, and close with the remaining calibration limits.

2. Scope and Contributions

Table 1 states the scope used throughout the paper. Each row connects one claim to the evidence used for it and to the stronger statement that the paper does not assert.

Table 1 Claim scope and evidence.

Claim	Evidence used here	Boundary
Robust candidate MaxWeight stability	Main-text proof, electronic-companion derivation, and Lean checks for exact and approximate oracle versions.	Holds only under the stated slack, lower-service, penalty, oracle, and moment conditions.
Variable-duration trajectory stability	Frame-level proof and Lean checks for cumulative service, bounded reconfiguration loss, duration bounds, and embedded recurrence.	A migration or port trace enters this theorem only after its cumulative stochastic service and moment model is certified.
Measured finite service domains	Declared q00/q01/q10/q11 service slices, mixed-portfolio replay, lower-service slack certificates, and probe/defer status for unknown states.	Does not imply positive service for unmeasured workloads, hardware states, or arbitrary future arrivals.
Scheduleurm theorem objects	Measured service rows, candidate configurations, admission boundaries, penalty records, oracle-gap checks, and optional live-hook fields.	Does not make every production dispatch a theorem-grade global MaxWeight action.
Controlled and production evidence	Controlled launched/completed batches, strict completed-active production history, read-only shadow traces, and production readiness checks.	Supports admitted rows and controlled launches; raw telemetry, attempted-only rows, and future unknown jobs remain outside the theorem population.
External scheduler comparison	Registered policy-semantics baselines evaluated on the same measured service cache, plus scoped adapter or same-host runtime probes when available.	Does not claim direct full-stack superiority over every external scheduler or original multi-node deployment.
Cross-domain trajectory portability	Frozen FJSP integer ledgers, MMRCPSP renewal-stream and class-balance arrival-tape protocols, and separated port source-core and synthetic-migration certificates with exact registered-family oracle audits.	Does not claim arbitrary domain-solver dominance, industrial port validation, or automatic transfer of server stability.
Learning and hidden-regime extensions	Formal conditional results and finite active-bucket replay checks, forced exploration, switching, and detector-delay records.	Treated as extensions rather than main live performance or complete online-learning claims.

The table separates what is proved or measured from nearby claims that require additional experiments or assumptions.

3. Related Work

3.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Learning-based schedulers such as Decima and DL2 show another way to adapt scheduling decisions from traces or reinforcement learning (Mao et al. 2019, Peng et al. 2019). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). Scheduleurm follows the same empirical lesson. Its

q00, q10, and q11 buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

3.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004, Neely 2010). Foster-Lyapunov and fluid-limit arguments also underlie positive recurrence and moment control in multiclass queueing networks (Dai and Meyn 1995, Meyn and Tweedie 2009). Our proof follows this lineage and separates three implementation-facing losses within the approximate-MaxWeight drift argument: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates. The current experiment artifact closes the deterministic finite active-bucket union bound and replay dwell/switching accounting, and adds a concrete deployable probability model: deterministic round-robin forced exploration over the finite active bucket set and a bounded two-window mean-shift detector. The certificate covers 64 active buckets and 120 replay scenarios with minimum forced samples 1561 per bucket, Hoeffding radius 0.0523, detector union tail bound 0.00564, and detection-delay bound 1024 decisions. A 2026-06-12 optional live-state probe further verifies that the theorem-facing live hook emits active-bucket sampler and regime-detector records while preserving the lower-service theorem oracle. This remains an opt-in extension path; the default live scheduler policy is not changed.

3.3. Finite configuration trajectories in operations research

Several classical operations-research scheduling domains share the same finite configuration-trajectory structure without sharing the same physical service model. In the flexible job-shop scheduling problem (FJSP), a trajectory chooses an eligible machine and a position in the machine

sequence for every operation; the Kacem instances provide heterogeneous processing times for these choices (Kacem et al. 2002). In the multi-mode resource-constrained project scheduling problem (MMRCPSP), each activity chooses a duration/resource mode subject to precedence and renewable and nonrenewable resource constraints; the Project Scheduling Problem Library (PSPLIB) supplies standard instances (Kolisch and Sprecher 1997). In port scheduling, berth positions, specific quay cranes, crane travel, and setup times form another configuration trajectory; the Berth Allocation and Specific Quay Crane Assignment Problem with travel and setup times (BACASP-S) formulation and instances make those costs explicit (Correcher et al. 2024). Section 7.4 uses these domains only to test portability of the finite-action selector. It does not identify their arrival, service, or reconfiguration laws with those of a GPU cluster, and it does not transfer the server recurrence theorem without a domain-specific stochastic model.

4. Model

4.1. Notation

Table 2 fixes the main slotted-model and replay notation. Frame-specific trajectory symbols are defined at first use in EC.6. Each mathematical symbol is reserved for one meaning; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

4.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t. A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served, which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω, action a generates a nonnegative potential-service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. Potential service is the work that the configuration could supply under backlog; it may be wasted when a queue is empty. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use $\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

Let $\mathcal{F}_t$ be the pre-decision filtration generated by the history through slot t, including $Q(t)$ and scheduler-visible state $\chi(t)$. The feasible families, service tables, penalties, and selected action a_t

Table 2 Notation.

Symbol	Meaning
$\mathcal{I}$	finite set of job classes
i	job-class index
t	discrete decision slot
$Q_i(t)$	class-i backlog at slot t
$Q(t)$	queue vector at slot t
$\mathcal{F}_t$	pre-decision information available to the scheduler at slot t
λ	arrival mean vector
v	feasible service vector
a	configuration action
a_t	action selected at slot t
$\mathcal{A}^{\text{full}}$	full configuration-action family
$\mathcal{A}^{\text{cand}}$	fixed candidate-action family
$\mathcal{A}_t^{\text{cand}}$	statewise candidate-action family at slot t
$\mathcal{A}^{\text{meas}}$	measured finite action slice
Z_t	hidden service regime at slot t
$\chi(t)$	scheduler-visible state at slot t
ω_t	exogenous randomness at slot t
$A_i(t)$	class-i arrival at slot t
$S_i(t)$	class-i potential service offered at slot t
$D_i(t)$	class-i completed departure at slot t
$\mu^z(a)$	service mean of action a in regime z
$\mu(a)$	main-theorem service mean of action a
$\mu_t^{\text{L}}(a)$	conservative lower-service estimate for action a at slot t
$C(\mathcal{A})$	convex hull of service vectors generated by action family $\mathcal{A}$
$\Lambda(\mathcal{A})$	downward-closed load region of action family $\mathcal{A}$
$\text{Mix}(\mathcal{A})$	probability simplex over action family $\mathcal{A}$
x_a	stationary mixing mass assigned to action a
q	nonnegative support vector
$H_{\mathcal{A}}(q)$	support function of action family $\mathcal{A}$ in direction q
Φ	finite feature map for configuration actions
m	feature dimension
d_Φ	weighted fabric metric induced by Φ
w_r	weight of feature coordinate r
L	service Lipschitz envelope in the fabric metric
ρ	candidate-cover radius
π	projection from full actions to candidate actions
$K_t(a)$	switching cost of action a at slot t
$G_t(a)$	risk or interference penalty of action a at slot t
$P_t(a)$	total penalty $K_t(a) + G_t(a)$
P_0	additive penalty bound
β	queue-scaled penalty coefficient
θ_{prof}	dimensionless profile-penalty fraction in replay
δ	full-region slack
ϵ_{cand}	candidate-set support loss
ϵ_{est}	lower-service estimation loss
ϵ_{LCB}	normalized coordinate confidence radius
α_0	additive approximate-oracle error
α_1	queue-scaled approximate-oracle error
η	residual drift margin
B	second-moment bound
N	finite-set threshold
γ	negative-drift margin outside the finite set
V	quadratic Lyapunov function
ΔV	one-step Lyapunov drift
b	replay backlog count
k	measured co-location profile
$\mathcal{K}_i$	measured support envelope of profiles for class i
$k_i^\star(b)$	replay profile selected for class i at backlog count b
$\widehat{\mu}_i(k)$	measured aggregate service of class i at profile k
$\widehat{\mu}_i^{\max}$	best measured aggregate service for class i in the same envelope
$r_i(a)$	coordinate confidence radius for service of action a
s_i	positive service-unit scale for class i
C_s	uniform upper bound on the service-unit scales s_i
c_i	task-native work units in one canonical replay job of class i

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

are $\mathcal{F}_t$-measurable; arrivals and potential service are realized after the action is selected. Completed departure is $D_i(t) = \min\{Q_i(t), S_i(t)\}$, and the queue dynamics are equivalently

$$\begin{aligned} Q_i(t+1) &= Q_i(t) - D_i(t) + A_i(t) \\ &= [Q_i(t) - S_i(t)]^+ + A_i(t), \qquad i \in \mathcal{I}. \end{aligned} \tag{1}$$

Here $A_i(t)$ is the class-i arrival in slot t. In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to impose the conditional mean and second-moment bounds below with respect to $\mathcal{F}_t$.

4.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \mathrm{Mix}(\mathcal{A}) \right\}, \tag{2}$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \le v \right\}. \tag{3}$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \qquad q \in \mathbb{R}_+^{|\mathcal{I}|}. \tag{4}$$

Here $\mathrm{Mix}(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a. The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \mathrm{Mix}(\mathcal{A}^{\mathrm{full}})$ such that $\lambda_i + \delta \le \sum_a x_a \mu_i(a)$ for every i. Consequently,

$$q^\top \lambda + \delta \, \|q\|_1 \le H_{\mathcal{A}^{\mathrm{full}}}(q), \qquad q \ge 0. \tag{5}$$

Inequality (5) is the bridge from capacity to MaxWeight drift; it is not an operational stability "if and only if" by itself.

4.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\text{full}} \to \mathbb{R}^m$ and weighted ℓ_1 fabric metric

$$d_\Phi(a, a') = \sum_{r=1}^{m} w_r |\Phi_r(a) - \Phi_r(a')|. \tag{6}$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\text{cand}}$ is a ρ-cover of $\mathcal{A}^{\text{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\text{cand}}$ with $d_\Phi(a, \pi(a)) \le \rho$. If $\|\mu(a) - \mu(a')\|_\infty \le L d_\Phi(a, a')$, then

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1, \qquad q \ge 0. \tag{7}$$

The support-loss bound (7) is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ. For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

The next result makes the candidate construction explicit rather than taking a covering family as a primitive. Let $\mathcal{J}_\Phi$ be a finite collection of active feature cells and let $\kappa : \mathcal{A}^{\text{full}} \to \mathcal{J}_\Phi$ assign each full action to a cell. For every active cell, choose one full-action representative $\text{rep}(j)$, and define

$$\mathcal{A}^{\text{cand}} = \{\text{rep}(j) : j \in \mathcal{J}_\Phi\}. \tag{8}$$

PROPOSITION 1 **(Finite-cell candidate construction)**. *Suppose every $a \in \mathcal{A}^{\text{full}}$ and its representative satisfy*

$$d_\Phi(a, \text{rep}(\kappa(a))) \le \rho, \tag{9}$$

and the service map is L-Lipschitz in d_Φ. Then the family in (8) *satisfies*

$$|\mathcal{A}^{\text{cand}}| \le |\mathcal{J}_\Phi| \tag{10}$$

and, for every $q \ge 0$,

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1. \tag{11}$$

Proof. The candidate family is the image of the active-cell set under rep, so its cardinality cannot exceed $|\mathcal{J}_\Phi|$. For each $a \in \mathcal{A}^{\text{full}}$, let $a' = \text{rep}(\kappa(a))$. By (9) and Lipschitzness, $\|\mu(a) - \mu(a')\|_\infty \le L\rho$. Nonnegativity of q gives

$$q^\top \mu(a) \le q^\top \mu(a') + L\rho \|q\|_1. \tag{12}$$

Taking the maximum over $a \in \mathcal{A}^{\text{full}}$ proves (11).

Proposition 1 separates two obligations. The feature partition and representative map are constructive algorithmic objects; the radius and service sensitivity remain calibration objects. Exact measured slices are the special case in which every active cell is represented exactly and $\rho = 0$. A nonzero empirical $L\rho$ claim additionally requires the perturbation profiling described in Section 7.

5. Robust Candidate MaxWeight

5.1. Policy

Let $\mu_t^{\mathrm{L}}(a)$ be a conservative lower-service estimate for candidate action a at time t. Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a, and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$a_t \in \arg\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \big\{ Q(t)^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a) \big\}. \tag{13}$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (13),

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \left\{ Q^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a) \right\} \\ &\quad - \left\{ Q^\top \mu_t^{\mathrm{L}}(a_t) - K_t(a_t) - G_t(a_t) \right\} \\ &\le \alpha_0 + \alpha_1 \|Q\|_1 . \end{aligned} \tag{14}$$

The oracle loss in (14) is paid from the same slack budget as candidate loss and lower-service error. Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

5.2. Main stability certificate

5.2.1. Assumptions We state the assumptions explicitly because each one corresponds to a measurable artifact in Scheduleurm. All constants below are nonnegative and are fixed over the horizon under consideration.

Table 3 is the proof spine used below. The numbered lemmas give the main-text derivation; EC.1–EC.5 expand the same support, lower-service, drift, recurrence, and operational-capacity chain so that the reader can check it without opening the Lean files. EC.6 treats the separate variable-duration result, EC.7 records the conditional learning and hidden-regime extensions, and EC.8 gives the paper-to-Lean crosswalk.

Table 3 Proof roadmap for Theorem 1.

Stage	Mathematical role	Term consumed
Capacity support	Convert full-action slack into a support-function margin.	None.
Candidate cover	Compare the full action family with the candidate family.	ϵ_{cand}, equal to $L\rho$ when a fabric metric is used.
Lower service	Replace unknown mean service by conservative service rows.	ϵ_{est}.
Penalty and oracle	Account for switching, risk, and approximate optimization.	$P_0, \beta, \alpha_0, \alpha_1$.
Drift margin	Combine the preceding losses into the residual slope η.	Requires $\eta > 0$.
Foster step	Use the quadratic Lyapunov inequality and second-moment bound to obtain negative drift outside a finite set.	B, N, γ and the local-return condition.

The table shows why each assumption is needed. Lean checks the algebraic implications; service, cover, oracle, and moment conditions are supplied by the measured Scheduleurm certificates.

ASSUMPTION 1 **(Load slack)**. *The arrival mean vector λ has full-action support slack $\delta > 0$:*

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q), \qquad q \in \mathbb{R}_+^{|\mathcal{I}|}. \tag{15}$$

This is the support-function form of belonging to the interior of the full-action capacity base. It is a sufficient stability hypothesis; the operational necessity direction requires a separate conservation law.

ASSUMPTION 2 **(Candidate cover and lower-service error)**. *For every scheduler-visible state t and every $q \in \mathbb{R}_+^{|\mathcal{I}|}$,*

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 , \tag{16}$$

and

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + \epsilon_{\text{est}} \|q\|_1 . \tag{17}$$

When the fabric metric in Section 4.4 is certified, $\epsilon_{\text{cand}} = L\rho$. For an exact measured finite slice, $\epsilon_{\text{cand}} = 0$.

PROPOSITION 2 **(Diagonal-scaled lower-confidence support loss)**. *Let $\widehat{\mu}_i(a)$ estimate $\mu_i(a)$, let $r_i(a)$ be a coordinate confidence radius, and let $s_i \ge 0$ be a fixed service-unit scale. On the simultaneous confidence event*

$$|\widehat{\mu}_i(a) - \mu_i(a)| \le r_i(a), \qquad 0 \le r_i(a) \le \epsilon_{\text{LCB}} s_i, \tag{18}$$

for every admitted $a \in \mathcal{A}_t^{\text{cand}}$ and class i, define

$$\mu_{t,i}^{\text{L}}(a) = \widehat{\mu}_i(a) - r_i(a). \tag{19}$$

Then $\mu^{\mathrm{L}}_{t,i}(a) \le \mu_i(a)$ *and, for every* $q \ge 0$*,*

$$H_{\mathcal{A}_t^{\mathrm{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\mathrm{cand}}} q^\top \mu_t^{\mathrm{L}}(a) + 2\epsilon_{\mathrm{LCB}} \sum_i s_i q_i. \tag{20}$$

If $s_i \le C_s$ *for every* i*, Assumption 2 holds with*

$$\epsilon_{\mathrm{est}} = 2\epsilon_{\mathrm{LCB}} C_s. \tag{21}$$

Proof. The upper side of (18) gives $\widehat{\mu}_i(a) - r_i(a) \le \mu_i(a)$. Its lower side and the radius bound give

$$\mu_i(a) \le \widehat{\mu}_i(a) + r_i(a) \le \mu^{\mathrm{L}}_{t,i}(a) + 2\epsilon_{\mathrm{LCB}} s_i. \tag{22}$$

Multiplying by $q_i \ge 0$, summing over i, and maximizing over admitted actions proves (20). Finally, $\sum_i s_i q_i \le C_s \sum_i q_i = C_s \|q\|_1$, which yields (21).

ASSUMPTION 3 **(Penalty growth and approximate oracle)**. *Let* $P_t(a) = K_t(a) + G_t(a)$*. For all* $a \in \mathcal{A}_t^{\mathrm{cand}}$*,*

$$0 \le P_t(a) \le P_0 + \beta \|Q(t)\|_1 . \tag{23}$$

The selected action $a_t \in \mathcal{A}_t^{\mathrm{cand}}$ *satisfies the approximate-oracle condition*

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \left\{ Q(t)^\top \mu_t^{\mathrm{L}}(a) - P_t(a) \right\} \\ &\quad - \left\{ Q(t)^\top \mu_t^{\mathrm{L}}(a_t) - P_t(a_t) \right\} \le \alpha_0 + \alpha_1 \|Q(t)\|_1 . \end{aligned} \tag{24}$$

Exact robust MaxWeight is the special case $\alpha_0 = \alpha_1 = 0$*.*

ASSUMPTION 4 **(Stochastic domination and bounded second moments)**. *The action* a_t*, state-wise action families, lower-service map, and penalties are* $\mathcal{F}_t$*-measurable. For every class* i*,*

$$\mathbb{E}[A_i(t) \mid \mathcal{F}_t] \le \lambda_i, \tag{25}$$

and

$$\mu^{\mathrm{L}}_{t,i}(a_t) \le \mathbb{E}[S_i(t) \mid \mathcal{F}_t] \tag{26}$$

almost surely. Moreover,

$$\mathbb{E}\left[\frac{1}{2} \sum_i \{A_i(t)^2 + S_i(t)^2\} \,\middle|\, \mathcal{F}_t \right] \le B. \tag{27}$$

ASSUMPTION 5 **(Countable-state recurrence condition)**. *For a positive-recurrence conclusion, the augmented controlled state $X(t) = (Q(t), \chi(t))$ is a Markov chain on a countable state space. The set*

$$C_N = \{x : \|Q(x)\|_1 \le N\} \tag{28}$$

used by the drift certificate is finite and nonempty, and the corresponding closed class satisfies the usual local-return condition. Without these conditions we claim only the finite-set negative-drift certificate.

5.2.2. Deterministic slack accounting

LEMMA 1 **(Support slack after candidate and estimation losses)**. *Under Assumptions 1–2, for every t and $q \in \mathbb{R}_+^{|\mathcal{I}|}$,*

$$\begin{aligned}\max_{a\in\mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) \ge\ & q^\top \lambda \\ & + (\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}}) \|q\|_1 .\end{aligned} \tag{29}$$

Proof. By Assumption 1,

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q). \tag{30}$$

Assumption 2 gives

$$\begin{aligned}H_{\mathcal{A}^{\text{full}}}(q) &\le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 \\ &\le \max_{a\in\mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) \\ &\quad + (\epsilon_{\text{cand}} + \epsilon_{\text{est}}) \|q\|_1 .\end{aligned} \tag{31}$$

Rearranging proves the claim.

LEMMA 2 **(Selected lower-service margin)**. *Under Assumptions 1–3, for $Q = Q(t)$,*

$$\begin{aligned}Q^\top \mu_t^{\text{L}}(a_t) \ge\ & Q^\top \lambda \\ & + \left(\delta - \epsilon_{\text{cand}} - \epsilon_{\text{est}} - \beta - \alpha_1\right) \|Q\|_1 \\ & - (P_0 + \alpha_0).\end{aligned} \tag{32}$$

Proof. Let $M(Q) = \max_{a\in\mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^{\text{L}}(a)$. Since $P_t(a) \le P_0 + \beta \|Q\|_1$ for every candidate,

$$\begin{aligned}&\max_{a\in\mathcal{A}_t^{\text{cand}}} \{Q^\top \mu_t^{\text{L}}(a) - P_t(a)\} \\ &\quad \ge M(Q) - P_0 - \beta \|Q\|_1 .\end{aligned} \tag{33}$$

The approximate-oracle condition therefore implies

$$\begin{aligned} Q^\top \mu_t^{\mathrm{L}}(a_t) - P_t(a_t) \geq M(Q) - P_0 - \beta \|Q\|_1 \\ - \alpha_0 - \alpha_1 \|Q\|_1 . \end{aligned} \tag{34}$$

Because $P_t(a_t) \geq 0$, the left-hand side is at most $Q^\top \mu_t^{\mathrm{L}}(a_t)$. Applying Lemma 1 with $q = Q$ gives the stated inequality.

LEMMA 3 **(Quadratic one-step bound)**. *For $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and nonnegative vectors Q, A, S,*

$$\begin{aligned} V([Q - S]^+ + A) - V(Q) \\ \leq \frac{1}{2}\sum_i (A_i^2 + S_i^2) + Q^\top A - Q^\top S. \end{aligned} \tag{35}$$

Proof. It is enough to prove the coordinate inequality. For $q, a, s \geq 0$, $([q - s]^+)^2 \leq (q - s)^2$ and $[q - s]^+ \leq q$. Hence

$$\begin{aligned} ([q - s]^+ + a)^2 &\leq ([q - s]^+)^2 + a^2 + 2a[q - s]^+ \\ &\leq q^2 + s^2 + a^2 + 2q(a - s). \end{aligned} \tag{36}$$

Subtracting q^2, dividing by two, and summing over coordinates gives the result.

THEOREM 1 **(Robust candidate MaxWeight stability certificate)**. *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics* (1)*. Suppose Assumptions 1–4 hold. If*

$$\eta = \delta - (\epsilon_{\mathrm{cand}} + \epsilon_{\mathrm{est}} + \beta + \alpha_1) > 0, \tag{37}$$

then for $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] \leq B + P_0 + \alpha_0 \\ - \eta \|Q(t)\|_1 . \end{aligned} \tag{38}$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \leq \eta N \tag{39}$$

for some $\gamma > 0$, the process has a negative expected drift outside C_N. If Assumption 5 also holds, the standard countable-state Foster criterion gives positive recurrence of the corresponding closed communicating class.

Proof. Apply Lemma 3 with $(Q, A, S) = (Q(t), A(t), S(t))$ and condition on $\mathcal{F}_t$. Using Assumption 4,

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\le B + Q(t)^\top \mathbb{E}[A(t) \mid \mathcal{F}_t] \\ &\quad - Q(t)^\top \mathbb{E}[S(t) \mid \mathcal{F}_t] \\ &\le B + Q(t)^\top \lambda - Q(t)^\top \mu_t^{\mathrm{L}}(a_t).\end{aligned} \tag{40}$$

Write $D_t = Q(t)^\top \lambda - Q(t)^\top \mu_t^{\mathrm{L}}(a_t)$. Lemma 2 gives

$$\begin{aligned}D_t &\le P_0 + \alpha_0 \\ &\quad - \left(\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}} - \beta - \alpha_1\right) \|Q(t)\|_1 .\end{aligned} \tag{41}$$

Combining the two displays proves

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid \mathcal{F}_t] &\le B + P_0 + \alpha_0 \\ &\quad - \eta \|Q(t)\|_1 .\end{aligned} \tag{42}$$

If $B + P_0 + \alpha_0 + \gamma \le \eta N$, the drift is at most $-\gamma$ whenever $\|Q(t)\|_1 > N$. This is the finite-set Foster drift certificate. Under Assumption 5, the usual countable-state Foster theorem yields positive recurrence of the closed class.

COROLLARY 1 **(Finite-horizon expected-backlog bound)**. *Under the hypotheses of Theorem 1, suppose the Lyapunov variables are integrable. For every integer $T \ge 1$,*

$$\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] \le \frac{B + P_0 + \alpha_0}{\eta} + \frac{\mathbb{E}[V(Q(0))]}{\eta T}. \tag{43}$$

Consequently,

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] \le \frac{B + P_0 + \alpha_0}{\eta}. \tag{44}$$

Proof. Take expectations in the conditional drift bound of Theorem 1, sum from $t = 0$ to $T - 1$, and apply the tower property. The Lyapunov terms telescope and $V \ge 0$, so

$$\begin{aligned}\eta \sum_{t=0}^{T-1} \mathbb{E}[\|Q(t)\|_1] &\le T(B + P_0 + \alpha_0) \\ &\quad + \mathbb{E}[V(Q(0))].\end{aligned} \tag{45}$$

Dividing by ηT gives (43); taking $T \to \infty$ gives (44).

COROLLARY 2 **(Statewise feasible families)**. *Let the full family, candidate family, true service, lower service, feature map, and penalty be indexed by the scheduler-visible state* $X(t) = (Q(t), \chi(t))$*:*

$$\begin{aligned} \mathcal{A}_t^{\text{full}} &= \mathcal{A}^{\text{full}}(X(t)), \quad \mathcal{A}_t^{\text{cand}} = \mathcal{A}^{\text{cand}}(X(t)), \\ \mu_t &= \mu(X(t), \cdot), \qquad \mu_t^{\text{L}} = \mu^{\text{L}}(X(t), \cdot), \\ \Phi_t &= \Phi(X(t), \cdot), \qquad P_t = P(X(t), \cdot). \end{aligned} \tag{46}$$

Here $\chi(t)$ *records scheduler-visible availability, pinned jobs, node health, and measured capacity boundaries. Suppose the load-slack, calibrated candidate-cover, lower-service, penalty, oracle, and stochastic-moment conditions hold pointwise for every admitted state* $X(t)$*, with the same constants* $\delta, \epsilon_{\text{cand}}, \epsilon_{\text{est}}, P_0, \beta, \alpha_0, \alpha_1, B$*. Then the drift bound and the finite-horizon backlog bounds in Theorem 1 and Corollary 1 hold unchanged. If Assumption 5 holds for the augmented state, the same finite-set recurrence conclusion follows.*

Proof. The proof of Lemmas 1–2 is pointwise in all six objects in (46). Conditioning on $\mathcal{F}_t$ fixes the realized full and candidate families, service maps, features, penalties, and action. The same inequality chain therefore gives the same lower-service margin and quadratic drift bound. Uniformity of the constants over the admitted statewise family permits the tower-property and Foster steps without introducing an additional state-dependent loss.

The Lean 4 theorem prover artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and the queue-indexed specialization of Corollary 2. In that specialization the full family, candidate family, true and lower-service maps, feature map, and penalty all depend on the integer queue snapshot. The general $\mathcal{F}_t$-conditional form above is the conventional augmented-state extension; its recurrence conclusion additionally uses Assumption 5. The artifact also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds and a finite-horizon telescoping backlog inequality. The paper theorem numbers are not Lean identifiers. The theorem crosswalk in EC.8 gives exact searchable Lean identifiers for every paper-facing formal statement, including Theorem 1, its statewise and finite-horizon consequences, candidate construction, diagonal scaling, variable-duration drift, action-union monotonicity, and the operational capacity boundary. Lean-only helper lemmas are indexed in the submitted artifact crosswalk rather than restated as additional paper claims. *Scope.* The theorem applies when the stated slack, lower-service, penalty, oracle-error, and moment assumptions are certified; it is not by itself a certificate for every scheduler dispatch.

5.3. Proof decomposition

The preceding lemmas isolate the mathematical obligations that must be certified by the scheduler. Proposition 1 constructs a bounded candidate family; Proposition 2 converts coordinate confidence radii into a support loss; Lemma 1 is the capacity and approximation layer; Lemma 2 is the robust oracle layer; Lemma 3 is the queueing layer; and Theorem 1 with Corollary 1 is their Foster-Lyapunov composition. The Lean formalization mirrors this separation. The operational necessity direction is kept separate: an "operationally stabilizes" predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from the definition of the predicate. EC.1–EC.5 reproduce this complete conventional proof chain and state the operational sandwich formally; EC.8 maps each component back to its Lean theorem.

PROPOSITION 3 **(Variable-duration robust frame drift)**. *Let n index decision frames, let the frame duration satisfy $0 < \tau_{\min} \le \tau_n \le \tau_{\max} < \infty$, and let $Q^{\mathrm{F}}(n)$ be the frame-boundary queue. Suppose conditional arrivals are bounded by $\tau_n\lambda$, the frame second-order term is at most B_{F}, and the full action family has support slack δ. Write $\overline{S}^{\mathrm{F}}(n) = \mathbb{E}[S^{\mathrm{F}}(n) \mid \mathcal{F}_n^{\mathrm{F}}]$. Assume the selected cumulative lower-service trajectory has combined candidate and estimation loss ϵ_{F}, approximate-oracle losses (α_0, α_1), and satisfies*

$$\begin{aligned} \tau_n H_{\mathcal{A}^{\mathrm{full}}}\left(Q^{\mathrm{F}}(n)\right) \le{}& \left(Q^{\mathrm{F}}(n)\right)^{\top} \overline{S}^{\mathrm{F}}(n) \\ &+ \tau_n(\epsilon_{\mathrm{F}} + \alpha_1)\left\|Q^{\mathrm{F}}(n)\right\|_1 \\ &+ \alpha_0 + P_n^{\mathrm{F}}, \end{aligned} \tag{47}$$

with penalty bound

$$P_n^{\mathrm{F}} \le P_0 + \tau_n\beta\left\|Q^{\mathrm{F}}(n)\right\|_1 .$$

Define

$$\eta_{\mathrm{F}} = \delta - \epsilon_{\mathrm{F}} - \alpha_1 - \beta. \tag{48}$$

If $\eta_{\mathrm{F}} > 0$, then the quadratic Lyapunov function satisfies

$$\begin{aligned} &\mathbb{E}\left[V\left(Q^{\mathrm{F}}(n+1)\right) - V\left(Q^{\mathrm{F}}(n)\right)\middle|\mathcal{F}_n^{\mathrm{F}}\right] \\ &\qquad \le B_{\mathrm{F}} + \alpha_0 + P_0 - \tau_{\min}\eta_{\mathrm{F}}\left\|Q^{\mathrm{F}}(n)\right\|_1 . \end{aligned} \tag{49}$$

If the augmented frame-boundary state is countable, its backlog sublevel sets are finite and nonempty, and the local-return condition holds, then any $N \in \mathbb{N}$ and $\gamma > 0$ satisfying

$$B_{\mathrm{F}} + \alpha_0 + P_0 + \gamma \le \tau_{\min}\eta_{\mathrm{F}}N \tag{50}$$

give a finite-set Foster recurrence certificate for the embedded chain. Moreover, elapsed physical time through frame n is at most $n\tau_{\max}$.

Proof. Apply the quadratic queue inequality to cumulative frame arrivals and service, then combine full-action support slack with the cumulative lower-service oracle and penalty bounds. Collecting queue-scaled losses gives (48)–(49); (50) supplies negative drift outside a finite sublevel set. EC.6 gives the complete unit-matched derivation and the duration-one reduction.

6. Scheduleurm Instantiation

6.1. Measured action slices

Scheduleurm is an existing scheduler for local and remote research jobs. The new algorithmic layer deliberately separates the theorem-facing policy from the legacy launcher. The theorem-facing policy builds global or statewise candidate families with a queue vector, lower-service vector, penalty units, and robust MaxWeight score. Service profiling, replay, baseline comparison, slack accounting, and oracle-gap checks are implemented outside the legacy scheduler. The live scheduler exposes opt-in hooks: a one-task placement scorer and a global-batch soft-hint path. The global-batch hook computes a bounded candidate configuration and returns only a preferred-node hint to the legacy launcher. Final placement therefore still passes through the existing resource and launch checks. This design lets us test the theorem-facing policy without making it the default production dispatcher.

Table 4 maps each measured workload bucket to the finite action slice used by the theorem-facing policy. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket. The table is therefore both an experimental summary and a definition of the declared finite slices used in the capacity and replay claims.

The replay policy uses a queue-adaptive robust MaxWeight penalty selector rather than a fixed co-location profile. For a workload with backlog count b, it first restricts profiles to a measured support envelope $\mathcal{K}_i$ and then chooses

$$\begin{aligned} k_i^\star(b) &\in \arg\max_{k\in\mathcal{K}_i} \left\{ b\,\widehat{\mu}_i(k) - \theta_{\text{prof}}\widehat{\mu}_i^{\max} k \right\}, \\ \theta_{\text{prof}} &= 0.10. \end{aligned} \tag{51}$$

Table 4 Declared measured finite action slices.

Bucket	Dominant service bottleneck	Measured feasible family	Excluded boundary	Use in replay and certificate
q00 light control	control-plane overhead and short-job fragmentation	profiles 1–13	profile 14	replay profile 13; legacy cap 1
q01 GPU-bound compute	accelerator co-location and memory pressure	profiles 1–8	none observed in this node bucket	replay profile 1; profiles 4 and 8 retained for support and LCB certificates; legacy cap 3
q01 CNN ResNet-50	convolutional training throughput and tail drain	profiles 1–7	profile 8 placement/stability boundary	static and rolling replay profile 5; LCB support profile 7; legacy cap 3
q01 LLM DistilGPT-2	transformer forward-pass concurrency and memory pressure	profiles 1–6 and 8	profiles 10–11 boundary rows	replay profile 8; sparse online arrivals drain at profile 1; legacy cap 3
q10 CPU-host bound	host-core and memory-bandwidth contention	profiles 1–9	profile 10	replay profile 8; legacy cap 9
q11 CPU/GPU coupled RL	coupled host and accelerator progress	profiles 1–5	profile 6 thread/placement boundary on node007	replay profile 5; mixed-portfolio support includes profile 5; legacy cap 5

A profile is robust feasible only after measured lower-service and admission checks. The boundary column records the first excluded profile when a boundary was observed. Lower confidence bound (LCB) certificates may retain support profiles that are not selected by the finite-batch replay policy.

Here $\mathcal{K}_i$ is the measured support envelope for class i, $\widehat{\mu}_i(k)$ is the measured aggregate service for class i at profile k, and $\widehat{\mu}_i^{\max}$ is the best measured aggregate service in the same envelope. Thus the replay candidate uses lower-interference profiles such as q01 profile 1 when they stay within the measured support slack, while support certificates retain actions such as q01 profiles 4 and 8 for capacity accounting. The optional statewise drain is lower-service guarded and no-upshift: it may only reduce a profile when the measured lower-service and waiting-backlog checks certify no loss relative to the base action. This implementation matches the approximate-oracle theorem: the support envelope preserves the drift action at large Q, and the finite-batch delay preference is a bounded profile penalty that is paid from the same slack budget as $K_t + G_t$.

Table 5 maps the theorem objects to the Scheduleurm implementation. Rows generated by the theorem-facing policy expose the robust MaxWeight objective directly. Rows generated by the live hook require the same candidate, service, penalty, and oracle-gap fields before they are used as theorem evidence.

6.2. Theorem evidence populations

Scheduleurm distinguishes three populations. The explicit task-list replay population is used for performance comparison. The completed-active production population is used for service-map and oracle-gap checks. Raw scheduler history is treated as telemetry because it contains cancelled, attempted-only, benchmark, and externally adopted jobs.

Table 5 Theorem-policy crosswalk in Scheduleurm.

Theorem object	Scheduleurm representation	Interpretation
Queue vector Q	backlog vector recorded for each replay or live slot	Defines the backlog weights in $Q^\top \mu^{\mathrm{L}}(a)$.
Candidate action $a \in \mathcal{A}_t^{\mathrm{cand}}$	finite placement, co-location, admission, and drain-rule configuration	Represents a global or statewise placement/co-location configuration for the theorem-facing oracle.
State index $\chi(t)$	alive nodes, pinned jobs, queued jobs, available-device state, and measured admission status	Selects the statewise candidate family without changing the fixed-family drift constants.
Lower service $\mu^{\mathrm{L}}(a)$	conservative measured-service row	Conservative service vector used in the drift inequality.
Admission and probe state	measured bucket, capacity boundary, lower-confidence status, and probe/defer flag	Determines whether a row enters the theorem-facing population or receives zero theorem service.
Penalty $K_t(a) + G_t(a)$	bounded switching, risk, and delay-control terms	Consumes bounded or queue-scaled slack.
Oracle error (α_0, α_1)	oracle-gap and support-gap reports	Measures approximate maximization loss relative to the theorem objective.
Live placement hook	optional one-task and global-batch scheduler hint	Integration point; theorem use requires candidate, service, penalty, and oracle-gap fields.

The theorem-facing rows expose robust MaxWeight score semantics explicitly; the live hook is an integration surface and not, by itself, a stability certificate.

The completed-active Scheduleurm-controlled production population has 3,437 mapped records out of 3,437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service bridge with $\alpha_0 = \alpha_1 = 0$. A longer live-state dry-run covers 128 placement slots and 767 candidate rows without writing the queue or launching tasks. A separate bounded q01/q11 SchedduleurmBench run exercises the robust lower-service hook on real tasks. These checks show that the theorem-facing fields can be generated from the scheduler state. They do not treat raw telemetry as a clean arrival stream.

7. Computational Evidence

7.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. The online-arrival experiment then runs nominal-load Poisson and bursty traces and a four-level load sweep at factors 0.50, 0.70, 0.85, and 0.95, with three random seeds for every hardware-local task set. The load scale is a common dedicated service reference. For d workload classes and R shared lanes, let m_i be class i's largest measured dedicated lower-service rate per lane. Equal time shares give $\lambda_i^\star = Rm_i/d$ in native work units per unit time; the arrival generator uses job intensity $r\lambda_i^\star/c_i$ at load factor r. Thus $\sum_i r\lambda_i^\star/(Rm_i) = r$: the workload classes jointly consume fraction r of the dedicated service reference. Loaded actions may enlarge this reference region.

The replay is a joint event simulation, not a sum of independent single-workload simulations. All workload classes assigned to one hardware scenario compete for the same physical CPU or GPU lanes. Before robust MaxWeight scoring, each class's residual work and every service coordinate are divided by the same fixed positive canonical-work scale c_i. The resulting coordinates are unfinished canonical jobs and canonical jobs per unit time; the replay selector uses these common coordinates. Changing the numerical task-native unit of one workload, together with its c_i, therefore cannot change an action ordering. This unit conversion is separate from the LCB error scales s_i in Proposition 2. A directional loaded action requires one resident job and one target job on the same lane. Evaluation uses a phase-aware task-native completion model: a fresh launch charges initialization once, the measured loop rate advances a common physical-work horizon, and checkpoint or final-output time is charged once at completion. Keep and migration continuations do not repay completed initialization; migration uses its measured checkpoint, transfer, staging, and resume delay instead. Actions for the same workload may have different probe horizons, but they must share the same physical work unit and are compared over the shortest naturally completed horizon. A loaded frame ends when its target completes; resident service is truncated at the resident's remaining work, and any unfinished resident returns to the queue. Migration likewise occupies one lane for its measured variable-duration frame.

For selection, a target batch of width $b(a)$ defines the canonical lower-service holding time $h_a = b(a)c_{i(a)}/\mu^{\mathrm{L}}_{i(a)}(a)$. With $n_i(a)$ required jobs of class i, the replay score is

$$F_Q(a) = \sum_i Q_i \frac{\min\{n_i(a)c_i, \mu^{\mathrm{L}}_i(a)h_a\}}{c_i h_a} - P(a). \tag{52}$$

The selector maximizes this score exactly over the visible feasible candidates. Only equal scores are ordered by h_a, profile, penalty, and action identifier. Every frame records the selected score, the feasible maximum, and their gap. Evaluation subsequently computes natural completion times from individual remaining work and phase overheads.

Table 6 compares the current candidate policy with the legacy Scheduleurm caps on 48 closed-batch traces. The metric is completion of all jobs in the task list, not the time to finish a single job. In the task names, convolutional neural network (CNN) denotes the vision workload family and large language model (LLM) denotes the language-model workload family.

Scope. Table 6 compares all-job completion on declared task lists using measured service rows.

Table 7 summarizes 477 online-arrival comparisons spanning nominal Poisson, nominal bursty, and Poisson/bursty load-sweep traces. These are the common completed subset of 480 registered

Table 6 Candidate policy versus legacy Scheduleurm on explicit task lists.

Hardware-local task list	Traces	Makespan	Mean flow	p90 flow
q00 node003	3	4.083979	5.994441	5.808406
q00 node005	3	4.087442	6.008666	5.792982
q01 jtl311linux	3	1.501980	2.364265	1.702092
q01 node007	3	1.153703	1.532013	1.273425
q01 jtl110gpu	3	1.207497	1.564309	1.312066
q01 jtl110gpu2	3	1.132329	1.499733	1.192372
q10 node001	3	2.422221	1.705248	2.301150
q10 node002	3	2.430438	1.698841	2.301980
q10 node003	3	3.398961	3.486260	3.421811
q10 node004	3	3.607959	3.713465	3.644578
q10 node005	3	3.398778	3.476316	3.420893
q10 node006	3	3.591276	3.692842	3.628476
q11 jtl311linux	3	1.027136	0.994510	0.900864
q11 node007	3	1.123543	1.217034	1.093424
q11 jtl110gpu	3	1.030952	0.995402	1.026782
q11 jtl110gpu2	3	1.035129	0.998460	1.024511

Ratios are legacy divided by candidate, so larger is better for Scheduleurm candidate. Each row is derived from the same causal schema-v3 replay as the online, SOTA-style, ablation, migration, and slack summaries. p90 is the 90th percentile of job flow time.

Table 7 Online replay distribution over 477 comparisons.

Metric	Geomean/mean	Median	Min	5%	95%
Legacy / candidate makespan	1.574753	1.141828	0.987746	1.000000	4.037528
Legacy / candidate mean flow	2.360104	1.912356	0.985094	1.000000	11.806295
Candidate mean backlog jobs	1.671077	1.014114	0.000000	0.028435	4.982385
Candidate max backlog jobs	7.155136	6.000000	1.000000	2.000000	15.000000

Legacy strictly Pareto-dominates the candidate in 5 online traces; those tail cases remain in the reported distribution. Policy-semantics and ablation dominance counts are reported separately in Tables 10 and 9.

online traces: 3 had an unfinished tail under at least one registered policy and are excluded from every finite completion-ratio comparison. The candidate itself left an unfinished tail in 3 registered hardware traces. Relative to legacy caps, the conditional geometric-mean ratios were 1.574753× for makespan and 2.360104× for mean flow. The minimum scenario-level ratios were 0.987746× and 0.985094×, respectively. The largest observed candidate queue backlog was 25.000000 jobs, and the largest time-weighted mean queue backlog was 10.466854 jobs. All ETA-dependent rows use task-native phase-aware natural-completion models; history fallback is excluded from the admitted population.

The ablation experiment changes support-only and delay-only scoring, bounded penalties, state-wise loaded actions, migration actions, and the profile tie-break rule. Table 9 evaluates 6 pre-registered removals on the same cache and common completed traces as the full policy. All ablations share its selector and job ordering, changing only the named component. Removing migration

Table 8 Online replay strata from the same hash-bound population.

Stratum	Comparisons	Makespan geomean	Mean-flow geomean	Makespan min	Mean-flow min
Arrival-family strata					
bursty	48	1.690726	2.189011	1.000000	0.997320
load_sweep	382	1.554702	2.356324	0.989411	0.985094
poisson	47	1.625284	2.582068	0.987746	0.995806
Quadrant strata					
q00	60	3.581818	3.467239	1.000000	1.017723
q01	120	1.022222	1.390034	0.998264	1.017150
q10	180	2.140149	5.159768	1.019958	1.492405
q11	117	1.003920	1.001036	0.987746	0.985094

Ratios are legacy divided by candidate on the common completed cohort. The load-sweep family contains both Poisson and bursty arrivals at the four registered load factors.

Table 9 Component ablations on the unified replay population.

Ablated policy	Comparisons	Makespan ratio	Mean-flow ratio	Dominance count
Delay-only score	525	1.225081	1.293975	13
High-profile tie-break	525	1.000000	1.000000	0
No bounded penalty	525	1.000000	1.000000	0
No loaded-action ledger	525	1.002058	1.035099	0
No migration actions	525	1.000000	1.000000	0
Support-only score	525	1.000000	1.000000	0

Ratios are ablated policy divided by the full policy, so values above one favor the full policy. Dominance count is the number of traces on which the ablation strictly Pareto-dominates the full policy.

leaves the complete decision sequence and results unchanged when no migration action is available. The full policy is Pareto-dominated in 13 ablation comparisons; the table reports all single-metric changes rather than treating a non-dominance result as evidence that every module improves every metric.

7.2. Policy-semantics replay baselines

The main comparison uses simple rules on the same visible state and measured action library. They prioritize target support, lower-service delay, interference avoidance, packing, or backlog-dependent resource use. The registered throughput-table and finish-time-fairness identifiers execute the same target-support rule and are displayed as duplicates. The delay rule minimizes a lower-service duration proxy; it implements neither a Gittins index nor a completion-time oracle. These rules were motivated by scheduling objectives studied in Gavel, Pollux, Sia, and co-location systems (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023, Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Their labels describe the implemented rules rather than attributing the external systems' complete algorithms to this replay.

We also register 11 non-adjacent scheduler families as finite actions in the Scheduleurm+external action union. This is the form needed by the candidate-set theorem: once an external policy family is represented by a lower-service row and a bounded penalty, it can be evaluated by the same robust selector. Several systems also have scoped same-host runtime checks, and Decima has a same-domain Spark directed acyclic graph (DAG) simulator check. These checks support traceability. They are not used as the central comparison because the paper's optimization claim is about measured configuration actions, not about deploying every external control plane under identical production conditions.

The multi-node check covers Scheduleurm's own measured history. Two GPU nodes were reachable during the read-only fabric probe. The resulting candidate set contained 18 lower-service rows and 626 disjoint robust-MaxWeight configurations, including a selected two-node configuration with $\alpha_0 = \alpha_1 = 0$. A separate scheduler-history study found 4,058 strict theorem-admitted launches, 4,047 completions, 35 workload domains, 13 nodes, three GPU nodes, and no strict unadmitted launched rows. This supports a Scheduleurm-native multi-node history claim. It does not require claiming that each external scheduler has been run through its original multi-node control plane.

Table 10 reports all 7 registered policy-semantics families over the hash-bound hardware-local replay population. The result is stated directly from the exported diagnostic: Scheduleurm is Pareto-dominated on at least one common completed replay trace. This does not imply that Scheduleurm wins each metric on each trace; the per-policy ratios and strict-dominance counts retain those distinctions.

Across all 3675 registered policy-trace comparisons, a baseline strictly dominates the fixed candidate in 88 cases. The quadrant counts are q00: 0, q01: 52, q10: 4, and q11: 32. Among those baseline wins, the largest candidate-to-baseline cost ratio is 1.063766 in q01, 1.077130 in q10, and 1.012406 in q11. On the common completed subset, the candidate has lower costs on both aggregate metrics against 7 of 7 registered rules. The individual trace results exhibit tradeoffs.

Figure 2 visualizes the same policy-semantics comparison as a four-quadrant cost frontier. Each panel normalizes a registered SOTA-style policy by the fixed Scheduleurm candidate: the horizontal coordinate is mean-flow cost divided by Scheduleurm's mean-flow cost and the vertical coordinate is makespan cost divided by Scheduleurm's makespan cost. Each point is a geometric mean over the common completed hardware-local traces in that quadrant. Thus the Scheduleurm policy is fixed at $(1, 1)$; points above or to the right are costlier on the corresponding aggregate metric, whereas points below or to the left favor the baseline. The label identifies the nearest aggregate rule

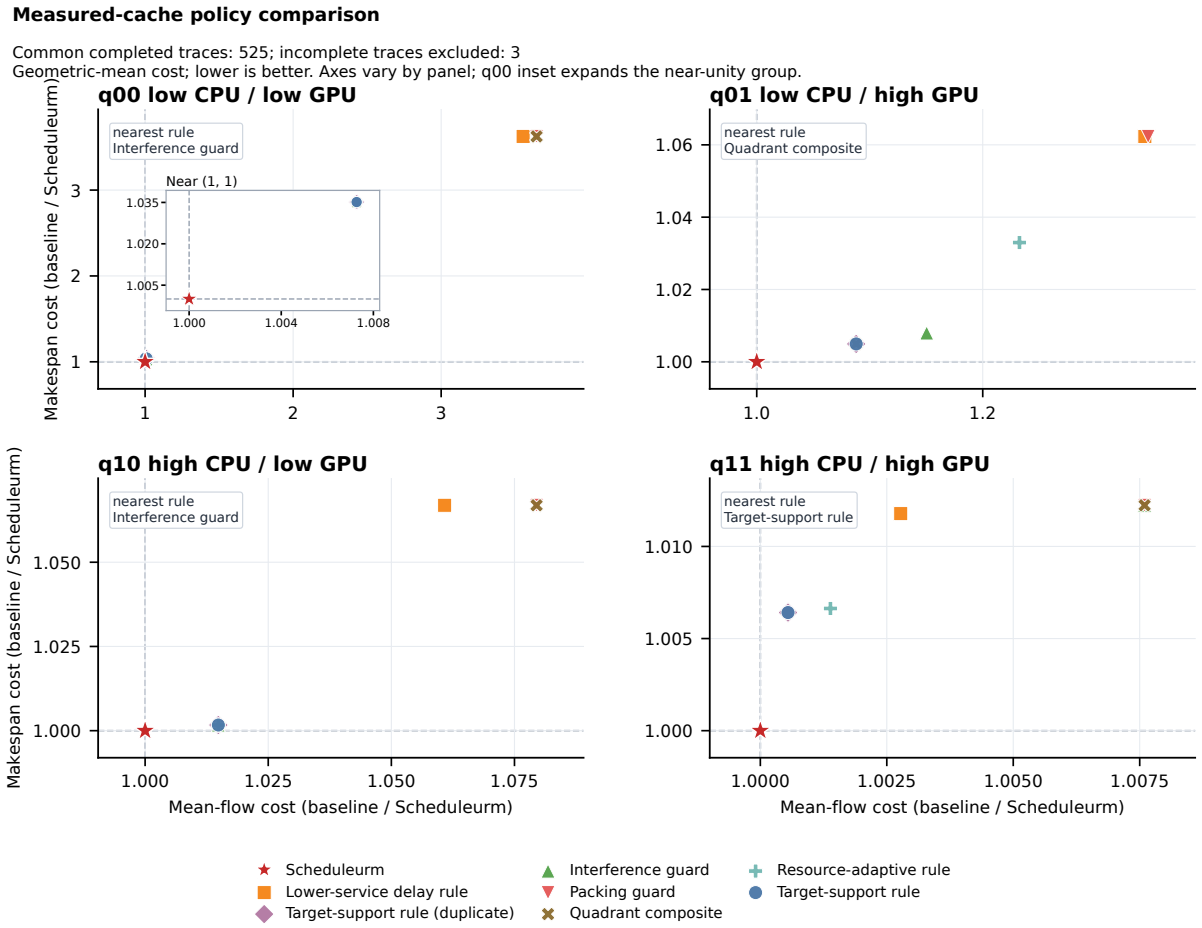


Figure 2 Four-quadrant measured-cache SOTA-style comparison. In each panel, the red star is Scheduleurm at $(1, 1)$. Other markers are registered SOTA-style policy families evaluated on the same task-native measured service cache, with x = normalized mean-flow cost and y = normalized makespan cost, both expressed as policy cost divided by Scheduleurm cost. Panel ranges differ; the q00 inset resolves the cluster near $1, 1$ while the main panel retains the full range. The box in each panel reports the nearest aggregate SOTA-style family under the maximum of the two displayed cost ratios. Points summarize geometric means and do not erase the trace-level baseline wins reported in Table 10. The comparison is a policy-semantics replay on shared measured service rows, not direct full-stack external binary execution.

under a minimax cost ratio. The figure is intentionally scoped to measured-cache policy-semantics comparison and is not a direct external binary benchmark.

We also evaluate the stronger theory-facing construction in which external policy-family actions are not merely baselines but are included in the candidate set. The selector evaluates the Scheduleurm candidate action together with the throughput-table, delay-oracle, interference-guard, finish-time-fairness, resource-adaptive, packing-guard, and measured bridge-action families under the same measured service units. The action is not deduplicated by co-location profile alone: a candidate is a finite configuration trajectory containing the placement/profile, resource partition, admission order,

Table 10 Policy-semantics aggregate on the identical measured service cache.

Policy-semantics baseline	Comparisons	Makespan ratio	Mean-flow ratio	Dominance count
Target-support rule (duplicate)	525	1.007814	1.028217	13
Target-support rule	525	1.007814	1.028217	13
Interference guard	525	1.010007	1.044527	6
Quadrant composite	525	1.209911	1.239238	12
Packing guard	525	1.226923	1.307439	15
Resource-adaptive rule	525	1.014865	1.061179	14
Lower-service delay rule	525	1.226784	1.292510	15

Entries are baseline divided by candidate, so values above one favor the candidate. Dominance count is strict trace-by-trace Pareto dominance with zero tolerance; aggregate ratios are geometric means and can favor the candidate even when some traces favor a baseline. These are policy-semantics rows, not full-stack external-system executions.

and statewise drain rule. This distinction matters on the ResNet-50 bucket, where the same profile supports different finish-time and interference-drain trajectories. The unified replay evaluates that finite Scheduleurm-plus-external action family directly instead of importing aggregate completion times from an older replay. For every registered trace, the selector enumerates only actions with a hardware-local lower-service row and applies the same canonical scaling, penalty accounting, lane feasibility, and variable-duration completion model. Table 10 therefore reports the action-union comparison on the same population as Tables 6–8; no separately normalized table of absolute times is mixed into the claim.

The strict measured-cache claim is separate from a direct-system claim. The final fixed candidate treats a configuration action as a measured trajectory, not merely as a co-location profile: the action identifies the hardware class, resource state, resident/target direction, admission order, and drain rule. Across q00, q01, q10, and q11, the exported trace-level comparison records whether any registered baseline Pareto-dominates that fixed candidate. The statement remains a measured-cache policy-semantics claim, not a direct full-stack superiority claim over external binaries.

The implementation layer includes the same action-union rule as an optional candidate policy. It adds a Pareto-slack Scheduleurm+external selector, state-dependent marginal service rows, bounded global batch lookahead, reusable online lower-confidence-bound (LCB) estimated time remaining (ETA) estimates, and external-policy action families for finish-time fairness, adaptive goodput, and packing guards. These additions are useful because the measured service curves are not monotone in load. In the controlled slice, a small CUDA task with a high video random-access memory (VRAM) resident job reached 1.095× the empty reference rate, and CPU-resident marginal rows reached 1.001×–1.091× the empty CPU reference. These rows expand the finite candidate family used by the theorem-facing replay; they are not used as evidence that the production scheduler already defaults to the optional global batch policy.

Candidate filtering uses only released jobs that are ready on the selected lane. The integer tail rule admits actions that leave this visible ready set drainable using the measured dedicated batch widths. If that set has no admissible action, the policy waits; the simulator advances to the next arrival or release event. The simulator's future event table never enters candidate filtering or scoring. Prefix tests vary the future arrival suffix while holding the observed history fixed and compare both candidate sets and selected actions. When the finite event source ends with an undrainable remainder, the run records its unfinished jobs and leaves all-job completion metrics undefined. This case occurs in the q11 action library, which has measured widths 2 and 5 but no singleton action. The exporter retains these failures and reports finite ratios on one common completed cohort for every baseline and ablation.

7.3. Theorem-condition and live-sanity checks

The hardware-local lower-service experiment instantiates the slack inequality in Theorem 1 separately on each of four GPU execution classes. For each class, the selected one-sided lower confidence bound (LCB) of a dedicated action is used as that workload's diagonal service scale. The four normalized workload coordinates therefore have unit dedicated service, and an equal stationary mix supplies $1/4$ normalized work units per slot to each coordinate. At load fraction 0.8, the registered Bernoulli model has

$$\begin{aligned} \delta &= \frac{1}{4} - \frac{0.8}{4} = 0.05, \\ \eta &= \delta - (L\rho + \epsilon_{\text{est}} + \beta + \alpha_1) \\ &= 0.050000. \end{aligned} \tag{53}$$

Here the candidate family is the exact finite set of measured dedicated actions, service is already the admitted one-sided LCB, and the finite oracle is exact and unpenalized. Thus the certificate records $L\rho = \epsilon_{\text{est}} = \beta = \alpha_1 = 0$ relative to that lower-service action family; it does not assert zero mean-service estimation error. The submission gate requires the exported minimum 0.050000 to be strictly positive. Table 11 is generated from the same cache hash as the replay. Its four rows are separate hardware-local stochastic models and do not pool nominally identical hosts.

The final service cache does not infer job completion time from a short steady-state window alone. Every admitted evaluation action contains a task-native phase-aware completion model with separate initialization, periodic work, and final checkpoint/output phases; no history-fallback row is admitted. Actions for one workload share a physical work unit and a common completed-work horizon. Empty and loaded rows are keyed by physical execution class, workload, profile, and

Table 11 Hardware-local lower-service stability certificates.

Node	Execution class	δ	η	LCB coverage
jtl110gpu	gpu_3080ti_12gb_dual:jtl110gpu	0.050000	0.050000	0.900000
jtl110gpu2	gpu_3080ti_12gb_dual:jtl110gpu2	0.050000	0.050000	0.900000
node007	gpu_2080ti_11gb_quad:node007	0.050000	0.050000	0.900000
jtl311linux	gpu_2080_8gb_dual:jtl311linux	0.050000	0.050000	0.900000

Each row uses four normalized Bernoulli arrival streams at load fraction 0.8. LCB coverage is nominal per node; cross-node simultaneous coverage uses the reported union-bound certificate. These rows certify exact measured dedicated actions. Loaded co-location actions remain statewise replay evidence rather than being silently absorbed into this empty-state stochastic model.

directional resident/target state. Consequently, rates from node007 are not borrowed for either jtl110 host or jtl311linux, and loaded target rates cannot overwrite empty-state legacy indices.

The same cache also re-evaluates the controlled migration ledger. Across 36 registered pure-CPU, pure-GPU, and hybrid-RL counterfactuals, migration is selected in 0 cases. The geometric ratios of without-migration to with-migration completion cost are 1.000000× for makespan and 1.000000× for mean flow. These ratios are reported even when they equal or fall below one: the admissibility rule charges checkpoint flush, synchronization, environment staging, resume warmup, lost work, and failure risk before selecting a move. Ordinary running user jobs are not migration candidates.

We also ran a corner-case marginal-efficiency experiment while the cluster was idle. Each probe used benchmark logs with explicit progress rate and estimated time remaining (ETA) lines. After three stable windows the probe stopped and the measured service row was available for replay. Table 12 reports the last stable marginal rate and the ratio to the corresponding empty resource baseline. The 30GB row holds a four-GPU resident memory job on the direct node007 GPU host and then adds a small CUDA task on one card. The CPU rows run 96-worker and 180-worker resident jobs on 192-core nodes and then add a small CPU task. These rows show that high resident video memory does not by itself imply a low marginal CUDA rate on this slice; on the idle 192-core CPU node, the single-thread marginal CPU micro-kernel also retained its empty-node rate under the 96-worker and 180-worker resident loads used here. Scheduleurm admits all stable positive-service rows in this controlled slice. Memory-packing policies admit the GPU rows when memory fits, and goodput policies admit rows whose loaded-to-empty ratio exceeds 0.90. The five stable canonical rows also have positive row-level lower-service slack at an offered load of 0.8 times the measured lower-service rate. This makes the corner-case rows theorem-facing for this finite measured slice without changing the default replay cache. Table 13 reports the same rows using job completion time (JCT) rather than marginal service rate.

A controlled live-dispatch check ran q01/q11 ScheduleurmBench tasks through the optional robust lower-service hook. A 6-task run emitted 7 theorem-grade dispatch slots and 13 candidate

Table 12 Controlled corner-case marginal service probes.

Scenario	Host bucket	Resident rate	Marginal rate	Loaded/empty ratio
CPU empty + small CPU	node001 CPU host	–	17.504	–
CPU 96-worker resident + small CPU	node001 CPU host	33.573	17.354	0.991
CPU 180-worker resident + small CPU	node001 CPU host	20.337	17.383	0.993
GPU empty + small CUDA	node007 GPU host	–	491.827	–
30.0GB GPU resident + small CUDA	node007 GPU host	1049.791	443.983	0.903

Rates are benchmark service units per second from explicit progress logs. The loaded/empty ratio compares the marginal row with the same small-task empty-resource baseline. The table is a controlled synthetic marginal-service slice, not a full-stack state-of-the-art performance claim or a production-wide theorem population.

Table 13 Gavel-style resident-delay/JCT holdout on the controlled co-location rows.

Scenario	Co-run mean JCT	Defer mean JCT	Mean JCT reduction	Makespan reduction
CPU 96-worker resident + small CPU	0.044	0.053	17.0%	29.1%
CPU 180-worker resident + small CPU	0.053	0.054	1.4%	30.4%
30.0GB GPU resident + small CUDA	0.217	0.322	32.5%	46.8%

Seconds are measured benchmark service-window seconds. The defer baseline uses the fastest observed resident-alone progress window, which favors defer. The table is a scoped measured-service holdout, not a direct Gavel full-stack execution.

rows. A larger 32-task run on an available two-GPU node emitted 32 theorem-grade slots and 56 candidate rows, completed all 32 tasks, and had $\alpha_0 = \alpha_1 = 0$. These runs check that the hook can launch and complete admitted tasks. They are separate from the larger production-history evidence.

The production evidence is based on strict scheduler history rather than on raw telemetry. Rows outside scheduler control, including attempted-only, auto-adopted, diagnostic, and read-only probe rows, are excluded. The admitted history contains 4,058 strict theorem-facing launches, 4,047 completions, 35 workload domains, 13 nodes, and no strict unadmitted launched rows. For closed shadow-subset snapshots, the theorem bridge has $\alpha_0 = \alpha_1 = 0$; CPU fallback slots use legacy sort-key semantics and are excluded from the theorem subset. The live evidence therefore supports controlled completion, strict scheduler-history completion, and read-only multi-node Scheduleurm coverage. It does not convert raw telemetry or future unknown jobs into theorem-facing arrivals. Table 14 summarizes the four evidence populations.

7.4. Finite-trajectory generalization beyond compute

The server action can be generalized without erasing domain physics by treating an action as a complete finite configuration trajectory. A trajectory records placement or mode choices, admissible

Table 14 Controlled and production evidence populations.

Population	Evidence	Interpretation
Controlled 6-task launch	6 launched and completed tasks; 7 theorem slots; 13 candidate rows.	Hook-level launch/completion check for admitted candidate rows.
Controlled 32-task launch	32 launched and completed tasks; 32 theorem slots; 56 candidate rows; $\alpha_0 = \alpha_1 = 0$.	Larger controlled live check on a two-GPU node.
Strict scheduler-history production	4,058 admitted launches; 4,047 completions; 35 domains; 13 nodes; no strict unadmitted launched rows.	Production-history evidence for admitted rows; raw 30-day telemetry is not the theorem population.
Completed-active service-map bridge	3,437 strict mapped records out of 3,437; mapped-slice slack 9.123×10^{-6}; service-map oracle bridge with $\alpha_0 = \alpha_1 = 0$.	Lower-service bridge for the controlled completed-active population.
Rolling queue snapshots	Active-production progress and closed shadow-subset oracle checks when available; CPU fallback slots excluded from the theorem subset.	Read-only monitoring and oracle conversion, not raw-history closure.
Live-oracle boundary	Large-scale live-oracle trace fields and global-batch soft-hint hook exist, but production-default global dispatch is not claimed.	Separates hook readiness from deployed theorem-grade production scheduling.

The populations are kept separate because they support different claims: controlled live execution, strict admitted-history completion, service-map bridging, read-only monitoring, and live-oracle readiness.

reconfiguration decisions, cumulative lower service, duration, and a bounded penalty. In each experiment below, the same selector exactly maximizes generated-family robust score

$$a_n \in \arg\max_{a \in \mathcal{A}_n^{\mathrm{traj}}} \left\{ \frac{\left(Q^{\mathrm{F}}(n)\right)^{\top} \underline{S}_n^{\mathrm{F}}(a) - P_n^{\mathrm{F}}(a)}{\tau_n(a)} \right\}, \tag{54}$$

Here n indexes decision frames, $Q^{\mathrm{F}}(n)$ is the frame-boundary backlog or urgency vector, $\mathcal{A}_n^{\mathrm{traj}}$ is the finite generated trajectory family, $\underline{S}_n^{\mathrm{F}}(a)$ is the cumulative lower-service vector, $P_n^{\mathrm{F}}(a)$ is the cumulative bounded trajectory penalty, and $\tau_n(a) > 0$ is the trajectory duration. For the FJSP and MMRCPSP renewal experiments, the artifact uses the actual duration ratio $\tau_n(a)/H_n$, where the positive normalization constant H_n is common to the frame. For each registered port context, every completed trajectory is instead idle-padded to a candidate-independent common horizon H_n; division by that common horizon preserves the scalar ordering. The duration factor is essential in the former case: maximizing the unnormalized cumulative numerator is generally not equivalent when candidate trajectories have different lengths. The port terminal coordinates are bounded virtual cost service, not physical vessel departures. The queue/urgency coordinates and feasibility oracle are domain-specific. Proposition 3 supplies the variable-duration stochastic interface, and the Lean theorem `frameApproximateOracle_iff_durationNormalized` checks the corresponding cumulative-oracle/rate-oracle equivalence for positive duration. The deterministic holdouts in this subsection test the candidate/oracle/penalty layer and do not by themselves instantiate a stochastic arrival model.

The following monotonicity result connects those experiments directly to the support-loss theorem. It distinguishes enlarging a registered action family from tuning the score after observing a holdout.

PROPOSITION 4 **(Finite action-union monotonicity)**. *Let* $\mathcal{A}^{\text{base}} \subseteq \mathcal{A}^{\text{union}} \subseteq \mathcal{A}^{\text{full}}$. *If, for every* $q \geq 0$,

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}^{\text{base}}}(q) + \epsilon \|q\|_1 , \tag{55}$$

where ϵ *is the certified base-family support loss, then the same inequality holds with* $\mathcal{A}^{\text{union}}$ *in place of* $\mathcal{A}^{\text{base}}$. *If* λ *has full-action support slack* δ, *the expanded family therefore satisfies*

$$q^\top \lambda + (\delta - \epsilon) \|q\|_1 \leq H_{\mathcal{A}^{\text{union}}}(q). \tag{56}$$

A ction-set inclusion gives $H_{\mathcal{A}^{\text{base}}}(q) \leq H_{\mathcal{A}^{\text{union}}}(q)$. Chaining this inequality with (55) proves the inherited support loss. Applying the full-action support-slack step used in Lemma 1 to that inherited gap gives (56); estimation, penalty, and oracle losses are deducted only afterward. The Lean theorems `support_gap_mono_under_candidate_expansion` and `candidate_capacity_slack_loss_under_expansion` check the two statements.

The FJSP evidence uses two chronologically separated protocols on Hurink edata, rdata, sdata, and vdata instances (Hurink et al. 1994). The first protocol compared the robust selector and five fixed dispatch rules using exact integer makespan and sum-completion ledgers. The fixed rules were first-in-first-out (FIFO), earliest completion time (ECT), shortest processing time (SPT), most operations remaining (MOR), and most work remaining (MWKR). A separate five-second, single-worker OR-Tools 9.14 Constraint Programming with Boolean Satisfiability (CP-SAT) run provided a makespan reference (Perron et al. 2023). Although the original family was nondominated against the fixed rules on all 20 instances, the CP-SAT trajectory dominated it on 14. This was an action-space failure, not evidence that a different score coefficient would repair the family.

We therefore froze a finite action-union policy before inspecting a disjoint 20-instance holdout. For each instance, the union contains the exact maximizer of the original generated family and the independently generated CP-SAT trajectory after feasibility verification; the robust score is unchanged. The exact union argmax selected the same trajectory as CP-SAT on 20/20 instances. Relative to the old family, it improved both coordinates on 18, was identical on 1, and gave one tradeoff on 1; the geometric-mean ratios were 0.9129 for makespan and 0.9117 for sum completion. This closes the registered action-space gap. It does not claim to beat CP-SAT after the CP-SAT

trajectory has become a member of the candidate family, and a feasible CP-SAT status remains distinct from an optimality proof.

The prospective structural MMRCPSP holdout registers 25 PSPLIB instances, five each from j12, j14, j16, j18, and j20 (Kolisch and Sprecher 1997). The selected candidate is Pareto nondominated against the frozen heuristic family on all 25 instances, strictly dominates every heuristic on 11, and attains the public makespan optimum on 7; its geometric-mean makespan ratio to the public optimum is 1.1667. These figures do not imply global or universal superiority. The subsequent stochastic experiment reuses the resulting known 25-class action library, where one action completes one project from one registered class. Seven exogenous unit-time arrival scenarios are replayed against duration-normalized robust MaxWeight, unnormalized robust MaxWeight, shortest registered trajectory, longest queue then shortest trajectory, and round-robin class selection. The first stream matrix uses three seeds; a later class-balance protocol freezes five new arrival tapes. The class-balance metric family was registered after observing the first matrix but before observing those new tapes. It is therefore a prospective arrival-tape check on a known instance and action library, not a new structural-instance holdout.

The port evidence likewise separates a failed policy restriction from its action-union revision. The first prospective BACASP-S source-core holdout used 54 instances from replications 83 and 84, with crane travel and setup times preserved (Correcher et al. 2024). The statewise robust policy was nondominated on only 27/ 54 instances. Against both SPT and reconfiguration-greedy, the baseline dominated on 27 instances, the robust policy dominated on 12, and 15 were tradeoffs. We retain this failure.

After observing it, we froze an 11-plan family before inspecting the disjoint replication-81/82 outcomes. The family contains five singleton policies and six fixed composite trajectories. Exact selection uses candidate-independent terminal virtual service and zero outer penalty, while each trajectory retains the audited low-level robust MaxWeight decisions. On the 54 new instances, the selected action was Pareto nondominated on 54 and strictly dominated all four baselines on 24; the remaining 30 tied the best SPT or reconfiguration-greedy action. Relative to either of those two strongest baselines, the geometric-mean ratios were 0.9659 for mean quay flow time, 0.9886 for quay makespan, and 0.9456 for source objective cost. There were no baseline-dominates cases on the action-union holdout.

The earlier registered port replay remains a separate deterministic frame check. Its locked 31-trajectory family gives 124 feasible trajectory-context ledgers and zero generated-family oracle gap

Table 15 Cross-domain finite-trajectory evidence.

Domain	Frozen protocol	Candidate result	Claim boundary
FJSP	Initial 20-instance diagnostic, followed by a disjoint prospective 20-instance action-union holdout; fixed-budget, single-worker CP-SAT	Initial CP-SAT dominance 14/20 retained. Expanded-family argmax matches the solver trajectory 20/ 20; versus the old family: 18 improvements, 1 identical, and 1 tradeoff.	Action-space closure, not CP-SAT or global FJSP superiority.
MMRCPSP structural holdout	25 prospective PSPLIB instances across j12, j14, j16, j18, and j20	Candidate nondominated 25/25 against frozen heuristics; strictly dominates every heuristic 11/25; attains public makespan optimum 7/25; geometric-mean makespan/optimum 1.1667.	Public optimum and CP-SAT reference are makespan-only; no global or universal superiority claim.
MMRCPSP aggregate queue	Known 25-class action library; seven scenarios, three seeds, and five policies; 105 runs	Exact robust oracle 21/21 scenario/seed cells, but shortest trajectory dominates it 21/21 on final backlog and time-average total queue.	Retained negative result; no total-delay superiority or recurrence claim.
MMRCPSP class balance	Same known library; seven scenarios, five new seeds, and five policies; 175 runs	Exact robust oracle and Pareto nondominance 35/35 on total time-average queue, worst-class time-average queue, and worst-class final backlog. Strictly better than every baseline 0/35; baseline dominance 0/35.	New arrival tapes, not new structural instances; no universal performance claim.
Port	Two disjoint 54-instance BACASP-S prospective holdouts: statewise policy on replications 83/84, then a frozen 11-plan action union on 81/82	Statewise policy nondominated 27/ 54; SPT/reconfiguration-greedy each dominate 27. Action union nondominated 54/ 54 and strict-all 24/ 54, with no baseline dominance.	Registered virtual-cost family only; no physical migration observation, stochastic port slack, industrial deployment, or global port optimum.

Every comparison is scoped to its registered finite policy family and stated cost coordinates. The rows deliberately retain comparator wins and do not assert domain-wide superiority.

over one source-core and three synthetic migration contexts. The frame certificate verifies a positive candidate-independent horizon and a compiled Lean frame interface. The terminal coordinates are virtual costs, not physical vessel departures; neither prospective holdout supplies stochastic port slack, observed industrial migration cost, or positive recurrence. Table 15 reports the resulting mixed evidence.

Three negative chains are especially informative. First, the earlier MMRCPSP instance-feasibility holdout exposed a renewable-resource error in the nonrenewable reserve dynamic program; its opened-row regression and disjoint repair holdout remain archived. Second, the new renewal-stream result shows that a valid exact MaxWeight oracle need not minimize aggregate delay: the shortest-trajectory baseline wins every aggregate-queue cell. The prospective class-balance result instead establishes a 35/35 nondominated tradeoff, not strict superiority. Third, the initial FJSP solver comparison and the port replication-83/84 holdout exposed candidate-family restrictions: an external solver trajectory dominated 14/20 FJSP ledgers, while SPT and reconfiguration-greedy each dominated 27/54 statewise port decisions. The subsequent action unions were frozen before their disjoint outcomes and removed those registered-family dominance cases. Proposition 4 explains why feasible expansion cannot worsen the support-loss certificate; it does not turn the revised

finite family into a universal optimum. This is consistent with the mathematics: the renewal-frame theorem controls drift under slack assumptions and is not a theorem of makespan or total-delay optimality.

The initial FJSP candidate ledgers and all qualitative CP-SAT relations reproduce on a second run; three exact gaps vary because a five-second wall-clock budget can return different incumbents. The disjoint solver-action holdout instead verifies the retained integer schedules and exact union argmax on every instance. Both MMRCPSP stream matrices reproduce exactly after decompression. The two port prospective artifacts bind compact certificates to their full ledgers and preserve the negative and revised protocols separately. The port source-core and synthetic migration semantics also remain separate: the former contains BACASP-S data but no observed migration, whereas the latter contains migration semantics but no physical terminal observations. These boundaries prevent portability evidence from becoming a domain-wide state-of-the-art claim.

7.5. Formal and production artifacts

The Lean proof artifact is submitted as a supplement. It contains the consolidated proof file, the Lean toolchain specification, the build log, a paper-theorem to Lean-theorem crosswalk, and checksums for the submitted files. The supplement build passes, and a comment-aware proof search finds no `sorry`, `admit`, or `axiom` proof terms. Full file hashes are reported in the supplement manifest rather than in the manuscript text.

For production data, the current theorem-facing population is the completed-active Scheduleurm-controlled set. It has 3,437 strict mapped records out of 3,437, with mapped-slice slack 9.123×10^{-6}. The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. By contrast, the raw 30-day history has 6,957 records, with 4,825 mapped records and 2,132 unmapped records, so it is not a clean theorem population. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw, attempted-only, external, or future scheduler row.

8. Discussion and Limitations

This paper connects a queueing-stability theorem with the objects available in a real heterogeneous scheduler. The main mathematical point is that candidate restriction is not treated as a heuristic shortcut. It is represented as an explicit support-function loss and paid from capacity slack together with estimation, penalty, and oracle losses. This gives a readable accounting rule: a scheduler may use a finite candidate family only when the remaining drift margin is still positive.

The finite-cell result gives a conditional compression bound. Given a finite feature partition, feasible representatives, certified radii, and service sensitivity, it bounds candidate cardinality and support loss. The current representative builder still scans its input candidates. Constructing those representatives within a budget K, without enumerating the full action space, and deriving the resulting $\epsilon_{\mathrm{cand}}(K)$ from the construction remains an algorithmic research question.

The Scheduleurm case study shows how this accounting can be implemented. The scheduler records measured lower-service rows, admissible configuration actions, capacity boundaries, penalties, oracle gaps, and admission status. The same records support four roles: performance replay, stability-condition calibration, production-history classification, and optional live-hook checks. This separation is useful because it prevents a strong empirical result in one population from being silently transferred to another.

The strongest empirical statement remains a measured finite-slice statement. For the declared service domains, exact measured actions give $\rho = 0$, and compressed candidate families report the corresponding nonzero $L\rho$. A future task is theorem-facing only if it is admitted to a measured domain with a positive lower-service row; otherwise it is assigned to a probe/defer state with zero theorem service. This is sufficient for the present measured-domain certificate, but it is not a universal positive-service theorem for all future workloads and hardware states.

The external-scheduler comparisons have a similarly precise role. They test whether the measured candidate family omits a useful scheduling semantic, such as goodput allocation, delay priority, or interference guarding. Once such a semantic is represented by a lower-service row and a bounded penalty, it becomes another finite action in the robust selector. This is the right comparison for the candidate-set theorem. It is narrower than a claim that Scheduleurm has beaten every external scheduler in its native full-stack deployment.

The production evidence should also be read at the correct level. Controlled live launches show that admitted theorem-hook actions can run to completion. Strict scheduler history shows that admitted production rows can be classified and completed at scale. Read-only shadow traces show that current queue states can be converted into theorem-facing candidate rows without mutating the queue. Together these checks support the measured Scheduleurm certificate. They do not turn raw telemetry, future unknown arrivals, or every live dispatch into a stability-certified arrival stream.

8.1. Limitations

The canonical score in (52) uses a lower-service holding time, while the evaluated frame ends at natural completion. Its zero oracle gap certifies maximization of that score. Transferring the

frame stability result requires a common conditional cumulative-service model, duration model, and arrival process for those same actions. The current dedicated-action Bernoulli certificates do not supply that model for loaded replay, tail waiting, or migration. Their four-node simultaneous LCB coverage lower bound remains 0.6; a global 95% guarantee requires a different statistical calibration.

Finite completion ratios condition on all policies completing and therefore do not establish superiority over the full registered arrival population. The q11 unfinished tails require measured singleton service or a separately declared batch-arrival model before full finite-stream completion can be guaranteed. The dedicated load reference is a declared service-mixture scale, not a certificate of the full state-dependent loaded capacity region. The present three-seed replay and finite-action construction also leave open independent confirmation and a computational budget-to-support-loss guarantee. These are substantive extensions beyond the current drift-accounting instantiation.

9. Conclusion

We studied heterogeneous compute scheduling as a configuration-action stochastic processing network. The resulting robust candidate MaxWeight theorem gives an explicit slack budget for candidate approximation, lower-service estimation, bounded penalties, and approximate optimization. If the residual margin is positive, the policy satisfies finite-horizon expected-backlog bounds and a finite-set Foster-Lyapunov recurrence certificate. A finite feature-cell construction bounds both candidate cardinality and the resulting $L\rho$ support loss, while diagonal scaling preserves the meaning of lower-confidence service across heterogeneous workload units. A separate variable-duration frame result extends the same slack accounting to cumulative configuration trajectories and recovers the slotted theorem at unit duration. Candidate-family expansion is support-loss monotone. The fixed-family, queue-indexed statewise, candidate-construction, action-union, diagonal-scaling, and frame statements are formalized in Lean.

Scheduleurm provides a concrete instantiation of the theorem. Its measured finite slices close the lower-service stability conditions for declared workload domains. On 48 closed-batch and 477 online replay comparisons, the robust candidate reports conditional legacy completion ratios and its trace-level relationship to every registered policy-semantics baseline is reported on the same measured service cache. Controlled live launches and strict scheduler-history evidence show how the theorem-facing objects can be produced by an operational scheduler. Frozen finite-trajectory experiments on FJSP, MMRCPSP, and port models further test the candidate/oracle/penalty interface. Their mixed

results include independent-solver and registered-policy counterexamples. Disjoint prospective action unions remove the corresponding registered-family dominance cases while retaining those failures. The evidence therefore supports portability of the interface without claiming domain-wide optimality or automatic transfer of the server stochastic model.

The contribution is therefore not a universal claim about every future workload or every external scheduler implementation. It is a mathematical and empirical template for making approximate configuration-action scheduling certifiable: define the action family, measure conservative service, account for every loss, and claim stability only when the residual slack remains positive.

Code and Data Availability

The submission package provides the Scheduleurm source, theorem-facing replay modules, measured service-cache summaries, generated experiment reports, and the Lean supplement headed by `ScheduleurmUpload.lean`. Its reproduction entry point `make reproduce-or-submission` recomputes the current replay from frozen local measurements, exports the common-cohort statistics and four-quadrant figure, aggregates the frozen cross-domain ledgers, and compiles the main paper and electronic companion. It performs no remote launches or measurement downloads. Lean compilation and historical evidence checks have separate commands documented with the artifact. Re-aggregating the cross-domain ledgers is distinct from rerunning their original solvers.

References

Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL http://dx.doi.org/10.1007/s11134-006-7586-8.

Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL http://dx.doi.org/10.1017/S0269964806060335.

Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL http://dx.doi.org/10.14778/3685800.3685845.

Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM), URL http://dx.doi.org/10.1145/3581784.3607060.

Correcher JF, Perea F, Alvarez-Valdes R (2024) The berth allocation and quay crane assignment problem with crane travel and setup times. *Computers & Operations Research* 162:106468, URL http://dx.doi.org/10.1016/j.cor.2023.106468.

Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL http://dx.doi.org/10.1287/stsy.2021.0081.

Dai JG, Meyn SP (1995) Stability and convergence of moments for multiclass queueing networks via fluid limit models. *IEEE Transactions on Automatic Control* 40(11):1889–1904, URL http://dx.doi.org/10.1109/9.471210.

de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, 625–635 (Springer), URL http://dx.doi.org/10.1007/978-3-030-79876-5_37.

Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL http://dx.doi.org/10.1287/stsy.2021.0087.

Hurink J, Jurisch B, Thole M (1994) Tabu search for the job-shop scheduling problem with multi-purpose machines. *OR Spectrum* 15(4):205–215, URL http://dx.doi.org/10.1007/BF01719451.

Kacem I, Hammadi S, Borne P (2002) Pareto-optimality approach for flexible job-shop scheduling problems: Hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation* 60(3–5):245–276, URL http://dx.doi.org/10.1016/S0378-4754(02)00019-8.

Kolisch R, Sprecher A (1997) PSPLIB: A project scheduling problem library. *European Journal of Operational Research* 96(1):205–216, URL http://dx.doi.org/10.1016/S0377-2217(96)00170-1.

Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL http://dx.doi.org/10.1145/3342195.3387547.

Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL http://dx.doi.org/10.1145/3529375.

Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL https://www.usenix.org/conference/nsdi20/presentation/mahajan.

Mao H, Schwarzkopf M, Venkatakrishnan SB, Meng Z, Alizadeh M (2019) Learning scheduling algorithms for data processing clusters. *Proceedings of the ACM Special Interest Group on Data Communication*, 270–288 (ACM), URL http://dx.doi.org/10.1145/3341302.3342080.

Meyn SP, Tweedie RL (2009) *Markov Chains and Stochastic Stability* (Cambridge University Press), 2 edition, URL http://dx.doi.org/10.1017/CBO9780511626630.

Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak.

Neely MJ (2010) *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Synthesis Lectures on Communication Networks (Morgan & Claypool Publishers), URL http://dx.doi.org/10.2200/S00271ED1V01Y201006CNT007.

Peng Y, Bao Y, Chen Y, Wu C, Meng C, Lin W (2019) DL2: A deep learning-driven scheduler for deep learning clusters. *arXiv preprint arXiv:1909.06040* URL https://arxiv.org/abs/1909.06040.

Perron L, Didier F, Gay S (2023) The CP-SAT-LP solver. *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *Leibniz International Proceedings in Informatics*, 3:1–3:2 (Schloss Dagstuhl–Leibniz-Zentrum für Informatik), URL http://dx.doi.org/10.4230/LIPIcs.CP.2023.3.

Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL https://www.usenix.org/conference/osdi21/presentation/qiao.

Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL http://dx.doi.org/10.1214/aoap/1075828046.

Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL http://dx.doi.org/10.1145/3600006.3613175.

Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL http://dx.doi.org/10.1109/9.182479.

Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL https://www.usenix.org/conference/osdi18/presentation/xiao.

Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL https://proceedings.mlr.press/v206/yang23d.html.

Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcca59228f2-Abstract.html.